Predicting When Training-Free Sparse Prefill Pays on Real Host-GPU Pairings: Research Gaps, Contributions, Methodology and Closest Prior Work

Mehul Dahiya · Research proposal brief · 18 October 2026

Research question. Can the end-to-end prefill time of training-free block-sparse attention be predicted for host-GPU pairings on which the workload has not been timed, using component measurements alone, and where does the prediction fail? Accuracy is not predicted. It is measured once, and the study tests whether attention recall can stand in for it.

- **RQ1.** Can measurements of the parts predict end-to-end prefill time, and the context length at which sparse starts to beat dense, on a pairing on which the workload has not been timed? No timing of the workload on that pairing goes into its prediction; the order of operations and the blocks each estimator chooses are recorded there only when its software path is new, with all timings thrown away.
- **RQ2.** Does keeping the order and position of each estimator's host work and synchronisations make that prediction better than models that only add up totals?
- **RQ3.** At the same recall in every layer, is accuracy the same whichever estimator chose the blocks?
- **RQ4.** Can those predictions decide, before the workload is timed on a pairing, whether to switch sparse prefill on, better than a decision based on kernel speedups alone?

Background. Block-sparse prefill skips attention blocks judged unimportant by an estimator. Papers usually report a kernel speedup on one machine. Our earlier measurements show that this kernel speedup often does not survive end to end. On an A100 with our first mask builder, end-to-end prefill became slower (0.63x to 0.75x) even though the sparse kernel beat PyTorch's SDPA flash kernel. Most of that builder's time was Python overhead: about 94% of the builder's loop time (a laptop profile at 32768 tokens, not banked; to be re-measured on the target host under the plan's null-kernel host-component profiler task). A faster builder turned the loss into a gain. Measured against the fastest correct dense kernel on that card, the gain is smaller still, and at 8192 tokens it becomes a loss. The kernel figures behind this are recomputed from measured kernel times; the end-to-end figures are arithmetic, not measurements, and are still to be measured. So host cost is not fixed: it depends on how an estimator is written. Even when masks are built on the GPU, released research code still makes the CPU wait for the GPU. XAttention, for example, checks GPU values from Python twice per query chunk in every layer. This is not just untidy code. Sparse attention keeps different blocks for every input, so an implementation must either ask the GPU how many blocks it kept, which makes the CPU wait, or reserve room for the worst case, which wastes GPU work, or keep a fixed number of blocks. Compiling the code does not simply remove the waits. We checked PyTorch's compiler (torch.compile) on four small examples: a check on a GPU value, reading a GPU number into Python, and finding which entries are non-zero each split the compiled graph, while picking a fixed number of top blocks did not. The study measures what remains on the GPU. Whether sparse prefill pays therefore depends on the host, the GPU, how their work overlaps and which waits an implementation makes, not on the kernel alone. This matters more on each new GPU: kernel-launch latency is almost flat across generations (about 6.1 microseconds on a V100, 6.0 on an A100 and 4.15 on an H100, IEEE LANMAN 2026) while attention kernels keep getting faster, and recent work reports sparse methods running slower than dense under chunked prefill (CompactAttention).

1. Research gaps

Gap	What is missing	Closest work and what it leaves open
G1	End-to-end latency on unseen CPU-GPU pairs can already be predicted with a learned model from host, transfer and device micro-benchmarks (CTFusion). It has not been tested whether an explicit host-GPU critical-path model predicts data-dependent sparse prefill on unseen pairings better than such a learned predictor, especially when only a few hardware systems are available for training.	CTFusion (IEEE FLTA 2025) predicts object-detection latency on unseen CPU-GPU pairs (17.6% error) with gradient-boosted trees trained on 20 systems; its host side is input preprocessing and the host-GPU interaction is learned, not modelled. nn-Meter, HELP and CloserToMe predict model latency on unseen devices, treating each device as one unit. KernelSight-LM assumes its host costs do not depend on the GPU but does not test this. None models data-dependent work and synchronisation inside a forward pass.
G2	No lightweight model follows the order and position, inside a prefill step, of each estimator's data-dependent host work and GPU-to-CPU synchronisations, wherever the masks are built, and predicts what they cost on a host-GPU pairing that has not been measured. Such a model also says what removing a synchronisation would gain before anyone writes a fused kernel. PyTorch's compiler does not remove these waits by default (see Background), so a compiled version of each estimator is also measured.	TaxBreak splits host time into framework, library and launch parts, and its HDBI is a ratio of sums; we adapt its null-kernel method to measure host costs, applying it to the whole sequence of operations rather than only the launch path, but it does not put those costs in execution order with the synchronisation points. TKLQT is an aggregate. KernelSight-LM, which we read in full, combines host and GPU time inside a step with one formula: GPU time, plus any excess of host time over it, plus a fixed tail (its Section 5.7, Eq. 4). Its event scheduler moves forward one batch step at a time (Section 4.2), so it does not record where in a step a synchronisation falls, and its synchronisation term is communication between GPUs. LLMServingSim 2.0 models synchronisation between devices, using profiles of each model and device. Fused GPU kernels such as VSPrefill's can remove host work and synchronisations, but released estimators often still make them. HiSparse, a decode-side system, fuses its per-layer work into one kernel that runs inside a CUDA graph without host-side branching, and is tuned by profiling each platform; it does not predict. None follows where host work and synchronisations fall within a prefill step, and TaxBreak notes its replay method can be imperfect for synchronisation-heavy kernels.
G3	It is untested whether attention recall (the share of the true attention mass the kept blocks capture) is a sufficient proxy for task accuracy across different estimators. We test this; we do not assume it. Prior work points both ways: within its own method, VSPrefill reaches full-attention accuracy above 90% recall, while Less Is More, in decoding, argues that which tokens are kept matters.	VSPrefill shows accuracy rising with recall on one dataset, and compares recall, not accuracy at the same recall, across selection methods. Less Is More treats recall as necessary but not sufficient in long decoding. The Sparse Frontier names the question as open. An oracle-guided prefill report (entry 26) lists a related check, how much of the oracle's choice a learned selector recovers, as future work. DFSAttn gives a lower bound on recall for one estimator family in video generation, but does not compare accuracy at the same recall. Whether two different estimators at the same recall give the same accuracy has not been tested.

Where the novelty lies. G2 and G3 are the main gaps, and both stay in scope. G1 is narrower: the problem of predicting latency on unseen pairs is already addressed by CTFusion, so G1 is kept as the validation question, whether an explicit model helps when only a few hardware systems are available for training. The measurement method is not claimed: host costs are measured by adapting TaxBreak's null-kernel method.

2. Contributions

Each contribution addresses one or more gaps. All predictions are written down before the measurements that test them, and failures will be reported as failures.

1. **A model of what data-dependent sparsity costs on each host and GPU (G2, with G1).** Because sparse attention keeps different blocks for every input, an implementation must make the CPU wait, reserve room for the worst case, or keep a fixed number of blocks. The model prices these choices on each pairing. It splits every operation into a host cost, a device cost and a link cost, and composes them in execution order with the synchronisation points of each estimator. It is compared against four alternatives built from the same measurements: an overlap-free sum; a fitted sum, in which the host, link and device totals each get one constant fitted on the pairings already run; a KernelSight-style step form that we build (an overlap floor plus a serial tail for synchronisations); and a learned predictor in the style of CTFusion (gradient-boosted trees on micro-benchmark features). The fitted sum is the main test, because it uses the same inputs with almost no fitting and no structure; the learned predictor is a secondary comparison, and its error is also reported when trained on one to six pairings, to show whether it is short of data. Like our model's, the fitted alternatives' predictions are saved before each pairing is timed, so they are trained only on the pairings already run; P2's are trained on P1 alone. Refitting them afterwards on all the other pairings is reported separately, as a look back. If a fitted alternative does better on unseen pairings, that is reported. A direct test of G2: XAttention is run twice, as released and with its Python asserts removed, and the model predicts the time difference on each pairing before it is measured. Each estimator is also run through PyTorch's compiler; on versions with no waits left, the model should do no better than the simpler forms. We do not claim the measurement method or the basic form of the model as new. Host costs are measured by adapting TaxBreak's null-kernel method to the whole sequence of operations, and letting the CPU run ahead of the GPU up to a queue limit is a standard model. What is new is placing each estimator's waits inside the prefill step and predicting their cost on hardware where the workload has not been timed.
2. **Validation on physically distinct hardware without timing the workload first (G1).** Seven real host-GPU pairings across two cloud providers, with five GPUs and four host CPU platforms. For each pairing the prediction is computed from component measurements, hashed and saved before the end-to-end workload is run. One pairing (H100 on an AMD host) is predicted from host and device costs measured on other pairings, with only its link costs measured on it, and one host platform (Sapphire Rapids) is held out as a forecast.
3. **A separability test for host and device costs (G1).** The measurements adapt TaxBreak's method; the test of whether they carry over is ours. A direct test of whether host costs measured with one GPU carry over to another GPU on the same host platform, and whether device costs carry over between host platforms for the same GPU. This decides when component costs can be reused across hardware. The same measurements test how the crossover point moves as the GPU gets faster relative to the host.
4. **A test of recall as an estimator-independent proxy for accuracy (G3).** Several estimators are compared at matched recall, matched layer by layer, at block sizes 32, 64 and 128, with results also reported for shallow, middle and deep layers. A control that picks blocks at random until it reaches the same recall shows whether recall alone carries accuracy, or whether the choice of blocks matters. If recall is sufficient, cheap recall measurement can replace expensive accuracy runs; if not, the result locates where dropped attention mass matters.
5. **A dispatcher that decides whether to switch sparse prefill on (application, RQ4).** For each pairing, context length and batch, it uses the predicted time and the lowest density that keeps accuracy to choose dense or a sparse estimator, before the workload is timed on that

pairing. It is scored by how much time its choice loses against the best measured choice, and compared with the decision a practitioner would make from kernel speedups alone (use sparse wherever the sparse kernel beats dense). If sparse never pays on a pairing, the dispatcher can at best match "always dense". Its value is then in avoiding the losses the kernel-speedup rule would make, and that is how it is reported. It decides in advance for each setting; it is not a live scheduler.

Scope. One GPU, one GPU stream per forward pass, batch sizes 1 and 4, and PyTorch with and without its compiler. The claims do not cover multi-GPU serving or serving engines that batch many requests together.

How the parts fit together. Sparse prefill pays only if it is faster and keeps accuracy. To predict that on a pairing where the workload has not been timed, we need the time at a given density (contributions 1 to 3) and the lowest density at which each estimator keeps accuracy. If recall turns out to be sufficient (contribution 4), that density can be read from recall, which is cheap and does not depend on the hardware, and profitability can then be predicted with no accuracy runs at all. If it is not sufficient, each estimator needs its own accuracy runs, and the study shows where recall falls short. Either way, the two halves form one prediction: when sparse prefill pays. The dispatcher (contribution 5) turns that prediction into a decision and scores it.

What is already in place

Item	Status
Earlier study on L4, A100 and H100	Done. Our first, CPU-side mask builder explains why kernel speedup did not survive end to end on the A100; a vectorised builder removes most of that cost. Two of three predictions registered before the H100 run passed.
Accuracy pilots on the L4 (Qwen2.5-1.5B, 16K and 32K tokens)	Done. With oracle masks, accuracy is non-inferior to dense only at 50% density, and only in two of the tested cells. The mean-pool estimator and XAttention with a single threshold were not non-inferior in any cell; XAttention with calibrated per-head thresholds is built and not yet run.
Measurement harness	Built and tested: one shared block selector and kernel for all estimators, commit-stamped rows, exact statistical analysis.
Component profilers, cost model, AWS tooling	To be built.

3. Methodology

3.1 Hardware pairings

Google Cloud fixes the host CPU by machine type, so the design crosses two providers. Three GPUs share one AMD host platform, and the L4 and the H100 each appear on two different hosts.

Pairing	Host CPU	GPU	Provider	Purpose
P1	Intel Cascade Lake	L4	GCP G2	Development pairing: the model is tuned and checked here, so it is not counted as a test.
P2	Intel Cascade Lake	A100	GCP A2	Same host platform as P1, different GPU.
P4	Intel Sapphire Rapids	H100	GCP A3	Host platform used nowhere else: forecast.
P3	AMD EPYC (3rd gen)	L4	AWS G6	Same GPU as P1, different host platform.
P5	AMD EPYC (3rd gen)	L40S	AWS G6e	Second GPU on the AMD host; faster GPU, same host.

P6	AMD EPYC (3rd gen)	H100	AWS p5.4xlarge	Same GPU as P4 on another host; host and device costs come from other pairings, and only its link costs are measured on it. Run last.
P7 (optional)	AMD EPYC (2nd gen)	A10G	AWS G5	Another host generation, if time allows.

Run order: P1, P2, P4, then P3, P5, P6 and P7. Every reused component still exists before the pairing that uses it. The Google Cloud pairings run first because Google Cloud sessions must start by 22 November 2026.

3.2 Measurement protocol for each pairing

1. **Environment.** The same container image on both providers; driver, CPU model, PCIe link, NUMA layout and clock state recorded on every row; threads pinned to the CPU node closest to the GPU; CPU frequency sampled throughout; GPU clocks and any throttling sampled during every timed block.
2. **Link costs.** Short microbenchmarks: host-device bandwidth from pinned and from pageable memory, also while the CPU runs a load the size of the largest mask build at each length; the fixed cost of each kind of synchronisation, measured with the GPU idle and the smallest possible transfer; and the depth of the GPU launch queue.
3. **Operation order.** The order of each estimator's operations and synchronisations depends on the software path, which can differ by GPU (XAttention uses Triton on the A100 and H100 and PyTorch elsewhere). It is recorded once per software path. For a path not seen before, each estimator is run once at two short lengths on the target and only the order is kept; all timings are thrown away. Costs are compared between pairings only when the order matches. The blocks each estimator chooses (its masks) are also recorded once per software path, without timing. On P2 both of XAttention's paths are run on the same GPU, using a switch the authors' code already has, to check whether they choose the same blocks. Masks are recorded on a new GPU only if that check fails.
4. **Host costs.** Each estimator's operation sequence with device work replaced by empty kernels, plus host-side computation (such as CPU mask building) run directly at the target size.
5. **Device costs.** Each estimator's whole sequence of GPU kernels replayed in order as a CUDA graph, with the masks recorded for each prompt. This keeps the GPU cache state between kernels and has no host work in it. Each operation is also replayed on its own as a cross-check, and the difference is reported as the cache effect. The same method is used for every estimator. Because TaxBreak warns that replay can be imperfect for synchronisation-heavy kernels, replayed kernel names and durations are first compared with the same kernels in a traced run on P1. The allowed gap is fixed in advance; if P1 exceeds it, the device-cost method is revised before any other pairing is run.
6. **Prediction saved.** Predicted end-to-end time computed, hashed and synced to storage before step 7. No timing or trace of the workload on the target goes into the prediction: only seconds of microbenchmarks there, plus the operation order and the masks recorded per software path with all timings thrown away.
7. **Ground truth.** Untraced end-to-end prefill for every estimator, length and batch size (1 and 4), with a dense control in every session. Every estimator runs at 2048, 4096, 8192, 16384 and 32768 tokens. The two short lengths are checks on the model, where CPU work and waits matter most, not settings anyone would use; no speedup claim is made there. The median and the 90th and 99th percentiles over repeats are reported.
8. **Profiler overhead.** Traced and untraced times are compared for every estimator. Traces are used only to check the model, never as inputs.

3.3 Cost model

For operation *i*, h_i is its host cost, d_i its GPU execution time and l_i its link cost (host-device copy time, zero for a kernel). A synchronisation is its own operation and is costed once: s_0 is a fixed cost for that kind of synchronisation, measured with the GPU idle, plus the size of any data it copies divided by the measured bandwidth. The waiting time is never measured as a constant; the model works it out. An `assert` on a GPU tensor counts as two operations: a small GPU kernel, then a synchronisation. Only device costs are carried between hosts; link and synchronisation costs are always measured on the target pairing. H_i is when the host has issued the operation and D_i is when the GPU finishes it:

$H_i = H_{i-1} + h_i$

if the launch queue is full (Q pending): $H_i = \max(H_i, D_{i-Q})$

if operation i is a synchronisation: $H_i = \max(H_i, D_{i-1}) + s_0$ and $D_i = H_i$

otherwise: $D_i = \max(H_i, D_{i-1}) + l_i + d_i$

predicted end-to-end time = D_N

The CPU and the GPU work at the same time: the CPU can queue up to Q operations ahead of the GPU and waits only at synchronisations, and the model follows exactly that. On the GPU side the model covers one stream, where copies and kernels run one after another; every estimator here uses one stream, and the traces are checked for this. Its claims do not extend to implementations that spread one forward pass over several GPU streams. Where the operation order puts a copy alongside CPU-side estimator work, the copy is costed at the bandwidth measured under CPU load on that pairing, never fitted and never taken from running the workload. Traces of the real run are checked afterwards, only as a check on the model. The four baselines use the same measurements: sum = Σh + Σl + Σd (no overlap), a fitted sum with one constant each for the host, link and device totals, fitted on the pairings already run when the prediction is saved, the KernelSight-style step form = max(Σh, Σl + Σd) per layer plus every synchronisation cost added at the end, and a CTFusion-style learned predictor trained the same way. Both are also refitted afterwards with one pairing left out at a time, and that look back is reported separately. The KernelSight-style step form uses the same synchronisation costs as our model but ignores where they fall, so beating it tests G2; beating the fitted sum tests G1, and the learned predictor is a secondary comparison. Stated in advance: on arms with no synchronisation inside the step (dense prefill, mean-pool, XAttention with its Python asserts removed, and compiled versions with no waits left), our model and the KernelSight-style step form should agree; an advantage is expected only for estimators with synchronisations. HDBI is reported alongside as a diagnostic. The model is also reported without its synchronisation term, and against host-only and device-only lower bounds, all from the same measurements. Two simple baselines are added: the same model with one constant host cost per host, and the measured time of the most similar pairing already run. For every prediction we report the error, whether it got the profitable-or-not decision right, the error in the crossover, and a prediction interval. The crossover is reported as the pair of neighbouring lengths between which sparse starts to beat dense, widened by the measurement uncertainty at each end. Every prediction is also recomputed with the mask density shifted by the difference measured between XAttention's two software paths.

Ablation. Each estimator is also run with its masks computed in advance and preloaded on the GPU. The difference from the live run separates the estimator's own cost and synchronisations from the kernel.

3.4 Hypotheses

#	Hypothesis	How it is tested

H0	Separability: host costs carry over between GPUs on the same host platform; device costs carry over between host platforms for the same GPU. (KernelSight-LM assumes the host half of this without testing it.)	Host side: P1 with P2, and P3 with P5 and P6. Device side: P1 with P3 (L4) and P4 with P6 (H100). Per operation, within a tolerance fixed in advance, and only where the operation order matches.
H1	Composition: on every test pairing (P2 to P7) the recursion predicts end-to-end time within tolerance, and on unseen pairings better than the saved predictions of all four alternatives, the fitted sum being the main one. The fitted alternatives are trained only on pairings already run. P1 is reported separately.	Relative error per pairing; sparse-versus-dense direction correct; for the two XAttention runs, predicted against measured time difference on each pairing.
H2	Transfer: predictions stay within tolerance when components come from other pairings, including P6, where both host and device costs come from other pairings.	Same statistic as H1; tested only if H0 holds.
H3	Forecast: on P4 the pair of neighbouring lengths between which each estimator starts to beat dense falls inside the interval predicted before the workload is timed on P4.	Predicted interval against the measured one, widened by bootstrap intervals; no crossover in range is reported as such.
H4	Recall sufficiency: at matched recall in every layer, accuracy does not depend on the estimator, at block sizes 32, 64 and 128. A random-selection control is held to the same recall.	Paired comparison against an equivalence bound fixed in advance and justified from the tasks; also reported separately for retrieval-like and local heads, and for the first, middle and last thirds of layers. Recall is matched only through each estimator's own settings (its threshold or budget), never by editing its masks, and the share of examples that find a match is reported. The comparisons of estimator pairs at the three block sizes form one family, corrected together (Holm); the head-type and layer-depth breakdowns are exploratory. The number of examples is set in advance from a power calculation: about 1,300 times the rate at which two estimators disagree on an example, at a 10-point bound and 90% power, and about four times that at a 5-point bound. That rate is taken from the upper end of its 95% range in the pilots, not its middle. A result without enough power is reported as inconclusive.
H5	Direction (secondary): on a fixed host, a faster GPU moves the crossover to longer contexts for estimators with host-side steps, less so for estimators that run fully on the GPU; on a fixed GPU, the cheaper host gives the earlier crossover.	Directions predicted from measured cost ratios before data (L4, L40S and H100 on the AMD host; the L4 and the H100 each on two hosts).
H6	Dispatch: on unseen pairings, at 16K and 32K tokens (where accuracy is measured), the dispatcher's choice of dense or sparse, per context length and batch, loses less time than the kernel-speedup rule and makes no more wrong calls.	Time lost against the best measured choice that keeps accuracy (dense included), per pairing, against a bound fixed in advance. Also compared with always dense, always the sparse estimator with the best kernel speedup, and the best choice on the most similar pairing already run. Every setting the dispatcher can choose is timed, so no extra sessions are needed.

Extension. On P5, prefill is also run in chunks (as serving systems do) for two estimators, and the extra selection cost per chunk is predicted before the run.

3.5 Estimators and accuracy evaluation

- **Random control for H4:** blocks chosen at random, with recorded seeds, until each layer reaches the target recall. It has no strategy of its own, so it shows whether recall alone is enough. It needs more blocks, so its density is reported too. It is used for accuracy only, not timed.
- **Estimators:** oracle (exact attention mass; a reference, not a guaranteed quality ceiling, since the mask that keeps the most attention need not be best for the task), sink-plus-local window (zero-cost floor), mean-pool (MInference style), vertical-slash style, XAttention with per-head calibrated thresholds (run as released, and again with its Python asserts removed; the second run loads only the authors' XAttention code with its asserts stripped, leaves the rest of the program and every lower-level check untouched, and counts only if its masks and outputs are identical to the first), and DuoAttention-style head masks where the authors released patterns. For each estimator we record whether its mask is built on the CPU or the GPU and how many times per layer it makes the CPU wait.
- **Compiled versions:** each estimator and the dense baseline are also run through PyTorch's compiler (torch.compile, using CUDA graphs where it can), with default settings and with the options that let it keep data-dependent sizes inside the graph. The authors' code is wrapped, not changed. We record where the compiler splits the graph and which waits remain. Compile time is kept out of the timings, and compiled and uncompiled versions are never mixed in one result.
- **Dense baseline:** chosen by a fixed procedure. The candidates are PyTorch's flash, cuDNN and memory-efficient kernels, FlashAttention-2, and FlashAttention-3 on the H100. A kernel is correct if it matches a float64 reference (or, where that does not fit in memory, agrees with another kernel; which check was used is named for each setting) and runs repeatedly without a GPU fault (cuDNN faulted above 8192 tokens on the L4 and A100). The fastest correct one, per GPU, length and batch, is chosen in a warm-up block before the timed runs, and every speedup is stated against it.
- **Recall:** at each layer, the share of that layer's dense attention mass, computed from its actual inputs in the same forward pass, that the kept blocks capture. Recall against a separate dense pass is reported as a second measure.
- **Models and tasks:** Qwen2.5-1.5B (primary), Qwen2.5-7B (recall and cost), Llama-3.1-8B (held out of all fitting); RULER tasks at 16K and 32K tokens, with a real-text haystack version planned.
- **Where quality is measured:** once, on the L4, with the dense control repeated on the A100. The A100 check passes if task scores agree within a bound fixed in advance; bit-for-bit agreement across GPUs is not expected. XAttention's thresholds are calibrated on the L4, where it runs its PyTorch code, while the A100 and H100 run its Triton code. The check on P2 above measures how much this matters, and accuracy results carry over to the Triton GPUs only if it passes.
- **Accuracy test:** an exact paired non-inferiority test (union-bound Clopper-Pearson), one-sided alpha 0.025, the same test used in the completed pilots. The main number reported is how much accuracy could at worst be lost (the lower confidence bound, in points). A margin of 10 points, with 5 points as a check, only decides which settings the dispatcher may use, and each margin is shown against the speedup it buys. The main cell uses at least 200 examples. A cell counts only if dense accuracy is above a floor fixed in advance.

3.6 Controls

Risk	Control

Different CPUs, caches, memory, PCIe	Host and link costs are measured on each real pairing, never simulated.
Boost clocks and virtual-machine noise	Cloud VMs cannot fix, and often cannot read, the CPU clock, and CPU frequency can change CPU-GPU timing in a non-monotonic way (Elango et al., IEEE INFOCOM 2026). So host speed is measured directly: a fixed CPU loop and a burst of empty GPU launches are timed at the start, middle and end of every session. A session is used only if this probe is stable within a bound, and costs are carried between sessions only if their probe results agree within a bound; both bounds are fixed in advance from repeat sessions. Threads pinned, warm-up, interleaved runs.
CPU mask building competing with copies for host memory	Copy bandwidth also measured while the CPU runs a mask-sized load; copies that overlap host-side estimator work use the loaded figure.
Differences between cloud providers	Same container image; before H2 is read, the same dense workload is run on P1 and P3 to measure provider-level variance, and H0 tolerances account for it.
A slow dense baseline making sparse look better	The dense kernel is chosen by the fixed procedure in 3.5, and our earlier A100 figures are restated against the fastest correct kernel. Its time in every timed block is compared with its warm-up time.
Cloud noise	The spread of end-to-end time between repeat sessions on P1, and within each session on every pairing, is reported as a number; prediction errors smaller than it are not interpreted. A bare-metal machine is beyond the budget, and the claims are about cloud hosts.
Cloud hardware changing over time	CPU microcode version, the provider's instance generation and how the machine was rented (on demand, spot, flex-start or reserved) recorded on every row, alongside the CPU model, so a later reader can tell whether the hardware changed.
Different software paths choosing different blocks	XAttention's two code paths are run on the same GPU on P2 and their masks compared; P4's masks are compared with P2's before P6 reuses them. Mask density is recorded on every row.
Different driver versions	Driver recorded on every row; host and device costs are compared only between sessions with the same driver, otherwise the comparison is marked as confounded.
GPU cache state lost when kernels are timed alone	Device costs come from replaying each estimator's kernels in order; timing them one at a time is only a cross-check.
GPU slowing down from heat or power limits	GPU clocks and throttling sampled in every timed block; affected blocks flagged and rerun.
Fitting the model to its own results	Predictions hashed and saved before the workload they predict is timed.
Profiler overhead	Ground truth is always untraced; traced-versus-untraced difference reported.
Compiler warm-up and recompiling	Compile time and warm-up are kept out of the timed runs; any recompile during a timed block flags it.
CPU-GPU placement and host memory type	Threads pinned to the GPU-local CPU node; pinned and pageable copies measured separately; any unified-memory page fault flags the row.
GPU launch queue filling up	Queue depth measured per pairing and included in the model; every prediction is also recomputed with the depth at one quarter to four times the measured value, and without the queue term, to show how much it matters.

Limits stated in advance. Seven pairings is still a small number, so results are reported per pairing and the claim covers only the measured hosts and GPUs. Batch sizes are 1 and 4 only. The recall study (G3) covers the tested models, tasks and block sizes, not every setting. The chunked-prefill extension reruns the estimator for each chunk and does not represent the best current chunked-prefill engineering. The main claim is predicting

time at a given density: in the pilots no deployable estimator matched dense accuracy at any tested density, so profitability for those estimators may turn out to be empty. If so, that is reported as a result, and the claims become the predicted against measured time and crossover on each pairing, which operations and waits keep each estimator from paying, and the recall result. Models that need several GPUs (70B parameters and above) are out of scope. Replaying the GPU kernels without the host may still miss some interactions between host and GPU work; the error in H1 measures how much this matters. If an estimator is rewritten so that it never makes the CPU wait, the model should predict no gain over simpler forms for it, and that is a correct prediction rather than a failure.

4. Closest prior work

Ordered from closest to our contributions to more distant, with further recent papers at the end. "Preprint" means not yet published in a peer-reviewed venue at the time of writing.

#	Paper	Venue	Year	Relation to this work
1	A Holistic Model for End-to-End Latency Prediction of Deep Learning Tasks on Heterogeneous Systems, CTFusion (Gennai, Ohsita and Shimonishi)	IEEE FLTA (Int. Conf. on Federated Learning Technologies and Applications)	2025	Predicts end-to-end latency on unseen CPU-GPU pairs from host, transfer and device micro-benchmarks with gradient-boosted trees. Closest to G1; our learned baseline follows it. Its workload is object detection with a preprocessing host side, not data-dependent work inside a forward pass.
2	TaxBreak: Unmasking the Hidden Costs of LLM Inference Through Overhead Decomposition (Vellaisamy et al.)	IEEE ISPASS	2026	Splits host overhead per kernel into framework, library and launch costs, using isolated replay, and defines HDBI; two H100/H200 systems. Closest to G2; does not model where synchronisations fall or predict across pairings. Our host-cost method adapts it.
3	KernelSight-LM: A Kernel-Level LLM Inference Simulator	arXiv preprint (2606.28565)	2026	Predicts serving latency on an unseen GPU; host constants measured per host and combined with GPU time as an overlap floor plus a serial tail. Closest to G1 and G2; dense attention only, no data-dependent work inside a step, and the host constants are assumed not to depend on the GPU. Our KernelSight-style step form, one of the baselines, is built from its step formula; it is our construction, not their simulator.
4	APEX: Asynchronous Parallel CPU-GPU Execution for Online LLM Inference on Constrained GPUs (Fan et al.)	IEEE IPDPS (Int. Parallel and Distributed Processing Symposium)	2026	Uses offline profiles to decide when CPU-offloaded decode attention should overlap GPU work, on two machines. Shows that moving a synchronisation point changes exposed time, which supports G2's premise; it schedules on profiled hardware and does not predict time on an unmeasured pairing.
5	nn-Meter: Towards Accurate Latency Prediction of Deep-Learning Model Inference on Diverse Edge Devices (Zhang et al.)	ACM MobiSys	2021	Kernel-level latency prediction on unseen edge devices. Closest learned-prediction precedent for G1; fixed model graphs, host and device as one unit.

#	Paper	Venue	Year	Relation to this work
6	HELP: Hardware-Adaptive Efficient Latency Prediction for NAS via Meta-Learning (Lee et al.)	NeurIPS	2021	Predicts latency on an unseen device from about 10 measurements. Related to G1.
7	CloserToMe: A Unified Framework for Accurate and Transferable Latency Prediction Across Heterogeneous Devices	AAAI	2026	Predicts latency of small benchmark networks on unseen GPUs, FPGAs and mobile chips from a hardware description and a few probe runs on the target. Related to G1; each device is one unit, with no host-GPU split.
8	LLMServingSim 2.0: A Unified Simulator for Heterogeneous and Disaggregated LLM Serving Infrastructure (Cho et al.)	IEEE ISPASS	2026	Serving simulator whose execution graphs include synchronisation between devices, built from profiles of each model and device. Related to G2; models serving systems, not data-dependent host work inside a prefill step, and does not predict an unprofiled pairing.
9	VSPrefill: Vertical-Slash Sparse Attention with Lightweight Indexing for Long-Context Prefilling	arXiv preprint (2603.04460)	2026	Accuracy rises with recall on one dataset; recall, not accuracy, compared across selection methods. Closest to G3.
10	Less Is More: Fast and Accurate Reasoning with Cross-Head Unified Sparse Attention (Yang et al.)	arXiv preprint (2508.07101)	2026	Treats recall as necessary but not sufficient in long reasoning decodes; on one problem, higher-recall settings still fail. Compares recall across methods, not accuracy at the same recall, and studies decoding, not prefill. Related to G3.
11	CompactAttention: Accelerating Chunked Prefill with Block-Union KV Selection	arXiv preprint (2605.16839)	2026	Under chunked prefill on two GPUs, XAttention's end-to-end time is 0.68x to 0.84x of dense speed, because of short query chunks and per-chunk mask search. An external reference for our chunked-prefill extension. Accuracy is compared at settings chosen per method, not at the same recall.
12	Characterizing and Optimizing LLM Inference Workloads on CPU-GPU Coupled Architectures (Vellaisamy et al.)	arXiv preprint (2504.11750)	2025	Three host-GPU systems. TKLQT adds up launch and queueing time per kernel to find where prefill stops being CPU-bound as batch size grows; host single-thread speed dominates at small batches. Host and GPU change together across its systems, and it does not predict time. Its Sapphire Rapids plus H100 launch figure (2.37 microseconds) is a second reference for our P4 check.
13	Vidur: A Large-Scale Simulation Framework for LLM Inference (Agrawal et al.)	MLSys	2024	Profile-based inference simulator, under 9% error, on the profiled GPU. Related to G1.
14	Hardware-Assisted Cross-Generation Prediction of GPUs Under Design	IEEE Transactions on Computer-Aided Design (TCAD)	2019	Predicts next-generation GPU performance. Prediction across hardware, but for GPU design, not host-GPU pairing.

#	Paper	Venue	Year	Relation to this work
15	The Framework Tax: Disparities Between Inference Efficiency in NLP Research and Deployment (Fernandez et al.)	EMNLP	2023	Shows that operation-count savings do not become wall-clock savings. The premise of this work.
16	The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs (Nawrot et al.)	arXiv preprint (2504.17768); ACL Findings 2026 reported, not shown in the paper	2026	Estimation cost limits sparse prefill. Reports FLOPs, not wall-clock time, and names as unknown how close a recall-maximising mask is to a task-optimal one, which is the G3 question. No host-device model.
17	MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention (Jiang et al.)	NeurIPS	2024	Source of the mean-pool and vertical-slash estimators; reports 5% to 30% of time spent building indices on one A100.
18	XAttention: Block Sparse Attention with Antidiagonal Scoring (Xu et al.)	ICML	2025	Calibrated-threshold estimator used as a baseline.
19	DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads (Xiao et al.)	ICLR	2025	Head-differentiated attention; used as a baseline for H4.
20	FlexPrefill: A Context-Aware Sparse Attention Mechanism for Efficient Long-Sequence Inference (Lai et al.)	ICLR	2025	Adapts sparsity per head and prompt; shows adaptive estimators are not new.
21	Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning (Lin et al.)	NeurIPS	2025	Adaptive sparsity budgets; related estimator design.
22	Improving the Performance of LLM Inference: Understanding the Role of CUDA Graphs (Franco et al.)	IEEE LANMAN	2026	Measures how CUDA Graphs remove launch overhead. Relevant to our device-cost measurement.
23	I/O-Aware PIM Acceleration for Long-Sequence LLM Inference with Hybrid Sparse Attention, PILOT (Lu et al.)	IEEE IPDPS	2026	Hardware for sparse prefill and decode. Orthogonal: new hardware, not host-GPU modelling.
24	iMLBench: A Machine Learning Benchmark Suite for CPU-GPU Integrated Architectures (Zhang et al.)	IEEE Transactions on Parallel and Distributed Systems (TPDS)	2021	CPU-GPU co-execution on integrated chips; our pairings are discrete PCIe systems.
25	Producing Wrong Data Without Doing Anything Obviously Wrong! (Mytkowicz et al.)	ASPLOS	2009	Measurement bias in systems evaluation; motivates our pre-registered protocol.
26	How Much Dense Attention is Necessary? Oracle-Guided Sparse Prefill for Full/GQA Layers in Hybrid Long-Context Models (Wang et al.)	arXiv preprint (2606.07703)	2026	Uses an attention-mass oracle as a reference and states it is not a global optimum. The per-query oracle stays within 1 point of dense on Qwen models, but support shared by 64 queries collapses at small budgets. Oracle masks are a premise, not our contribution. Data and code not released.

#	Paper	Venue	Year	Relation to this work
27	AB-Sparse: Sparse Attention with Adaptive Block Size for Accurate and Efficient Long-Context Inference	arXiv preprint (2605.12110)	2026	Decoding, not prefill. Recall against block size (16 to 64) differs by head and is stable across inputs; accuracy improves with recall at a fixed budget, but accuracy at the same recall is not tested. Block size is therefore a control in our study.
28	Token Sparse Attention: Efficient Long-Context Inference with Interleaved Token Selection	ICML	2026	Reports prefill latency on two GPUs (A100, A6000); an example of single-paper, two-machine evidence.
29	APEX: An Extensible and Dynamism-Aware Simulator for Automated Parallel Execution in LLM Serving (Lin et al.)	arXiv preprint (2411.17651)	2025	Serving simulator built from per-device operation profiles. Predicts speedups between parallel plans (10.7% average error), but its absolute per-token time is consistently below measured, which the authors attribute to leaving out operations other than the key ones. No host-side term. Supports G1; not used as a baseline.
30	HiSparse: Scaling Sparse-Attention Decoding with Hierarchical KV Cache Management (Xie et al.)	arXiv preprint (2608.07009)	2026	Decoding, not prefill. Keeps the full KV cache in host memory and a small cache on the GPU; one fused kernel per layer handles cache misses inside the decode CUDA graph, which needs no host-side branching. The best cache size is found by profiling each platform and depends mainly on host-link bandwidth (miss handling 112 microseconds per call over PCIe Gen5 against 29 over NVLink-C2C). Supports G2's premise that per-layer host-visible work and link speed decide end-to-end cost; it is a system design, not a predictive model.
31	DFSAttn: Dynamic Fine-grained Sparse Attention for Efficient Video Generation (Hu et al.)	ICML	2026	Video diffusion, not LLM prefill. Uses the same recall definition as ours and proves a lower bound: expected recall is at least the budget times the probability that pooled block scores pick the oracle's top blocks. Compares methods at different sparsities by output similarity to dense, not accuracy at the same recall. Related to G3; does not test whether recall is sufficient.

Also read in full and judged distant (supplied by the researcher): Kohli et al., IEEE HPEC 2025 (CPU, GPU and NPU benchmarking on one integrated edge chip); HyGIN, IEEE Computer Architecture Letters 2026 (CPU- versus GPU-initiated communication for multi-GPU training); Elango et al., IEEE INFOCOM 2026 (CPU-GPU frequency interaction in a 5G stack, used above as a confounder reference); PLA, IEEE TCAD 2026 (an accelerator with token pruning for point-cloud LLMs; its finding that attention concentrates with depth is why recall is also reported by layer depth); LLMEdger, IEEE ICFEC 2026 (pipeline and tensor parallelism scheduling for an encoder-decoder model on CPU-only edge nodes).

Withdrawn. SparKV (arXiv 2604.21231), listed in earlier drafts, has been withdrawn by its authors and is no longer cited.

How each entry was checked. Entries 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 26, 27, 29, 30 and 31 were read in full. Entry 29, a serving simulator also called APEX, is a different paper from entry 4. The others are described from their abstracts only. Each is being re-checked against its abstract, and all will be read in full before the related-

work section is written. Venues and years were checked by web search on 4 October 2026. Preprints (marked as such) whose peer-reviewed venue, if any, should be re-checked before submission. A search of IEEE Xplore and Scopus will follow before the related-work section is written.